ACADEMIC PROJECT PRESENTATION

Shader Program Compiler

2D and 3D Graphics Domain-Specific Language (DSL) Engine

A high-level declarative language and source-to-source transpiler compiling to native hardware-accelerated OpenGL 2.1 and SDL2 desktop binaries.

PROJECT METADATA AND SPECIFICATIONS

Author: Eldho | Semester 3 Mini Project (Compiler Design and Computer Graphics)

Technology: C++17 Transpiler, ANSI C Code Generation, OpenGL 2.1, SDL2, GLSL Shaders

Source Repository: https://github.com/eldhoGI2/shader-compiler

PROJECT OVERVIEW

The Problem and The Core Concept

The Problem: Raw Graphics Boilerplate

- **Extensive Setup Overhead**
  Creating a simple window and drawing a single shape in standard C requires over 50 lines of subsystem initialization and context management.
- **Manual State Management**
  Developers must explicitly manage projection matrices, modelview transformations, vertex arrays, and double-buffer swaps.
- **Shader Complexity**
  Writing and linking GLSL fragment shaders involves manual string loading, error log polling, program attachment, and uniform location caching.
- **High Cognitive Barrier**
  Low-level API details distract from graphics logic, animation algorithms, and interactive game design.

The Concept: Declarative DSL

- **Declarative Syntax**
  Developers specify what to render using concise, readable commands rather than writing procedural graphics initialization code.
- **Automated Compilation**
  A dedicated C++ compiler parses DSL scripts, generates clean intermediate C code, and automatically builds a native executable.
- **Abstracted Hardware Features**
  GPU fragment shaders, color dictionaries, affine transformations, and 3D camera matrices are handled automatically.
- **Integrated Game and 3D Modes**
  Activating physics and first-person 3D navigation requires only single commands: 'pong' and 'mode_3d'.

System Architecture: End-to-End Execution Pipeline



1. Frontend Lexer & Parser

• Reads .my source script and strips comment lines (#).
• Categorizes tokens: window, shapes, colors, shaders, pong, mode_3d.
• Performs syntactic parameter validation and constructs an internal Abstract Syntax Tree (AST).

2. Intermediate C CodeGen

• Translates AST nodes into intermediate ANSI C (generated_game.c).
• Injects the custom graphics engine header (#include "graphics_runtime.h").
• Compiles static draw calls into high-performance OpenGL display lists (glNewList).

3. Native Compilation & Link

• Automatically invokes GCC via system() without requiring manual Makefiles.
• Links system libraries: -lSDL2 -lGL -lGLU -lm.
• Produces final_game: a native standalone 60 FPS executable with full GPU acceleration.

CODEBASE ORGANIZATION

Project Structure: The Core and The Body

1. Compiler Core (compiler.cpp)

- Lexical Scanner: Recognizes 50+ token types for environment, shapes, 3D, and game logic.
- Syntax Parser: Validates parameters, coordinates, and color specifications.
- Code Generator: Emits generated_game.c and triggers GCC build command.
- Binary: Compiled into standalone compiler tool ./mycompiler.

2. Runtime Engine (graphics_runtime.h)

- OpenGL & SDL2 Subsystems: Window creation, context lifecycle, and event handling.
- GPU Shader Engine: Compiles and executes 8+ GLSL fragment shaders (plasma, grid, waves).
- Physics & Collision: 2D ball-paddle deflection and 3D wall-sliding collision math.
- Camera System: First-person 3D camera with mouse-look and keyboard traversal.

3. Domain Script Programs (*.my)

- demo.my: Initial baseline coursework proof-of-concept.
- showcase.my: 10+ 2D geometric shapes, 35+ named colors, and linear gradients.
- cyberpunk.my: GLSL fragment shaders, gradients, and transformation matrices.
- pong.my: Interactive 2D Pong game with ball physics, AI paddle, and score display.
- world3d.my: Navigable 3D first-person environment with obstacle collision.

4. Intermediates and Executables

- generated_game.c: Intermediate C source code produced by the transpiler.
- mycompiler: The C++ compiler binary used to translate scripts.
- final_game: The compiled native OpenGL desktop graphics executable.
- plasma.frag: External user-loadable GLSL fragment shader source file.

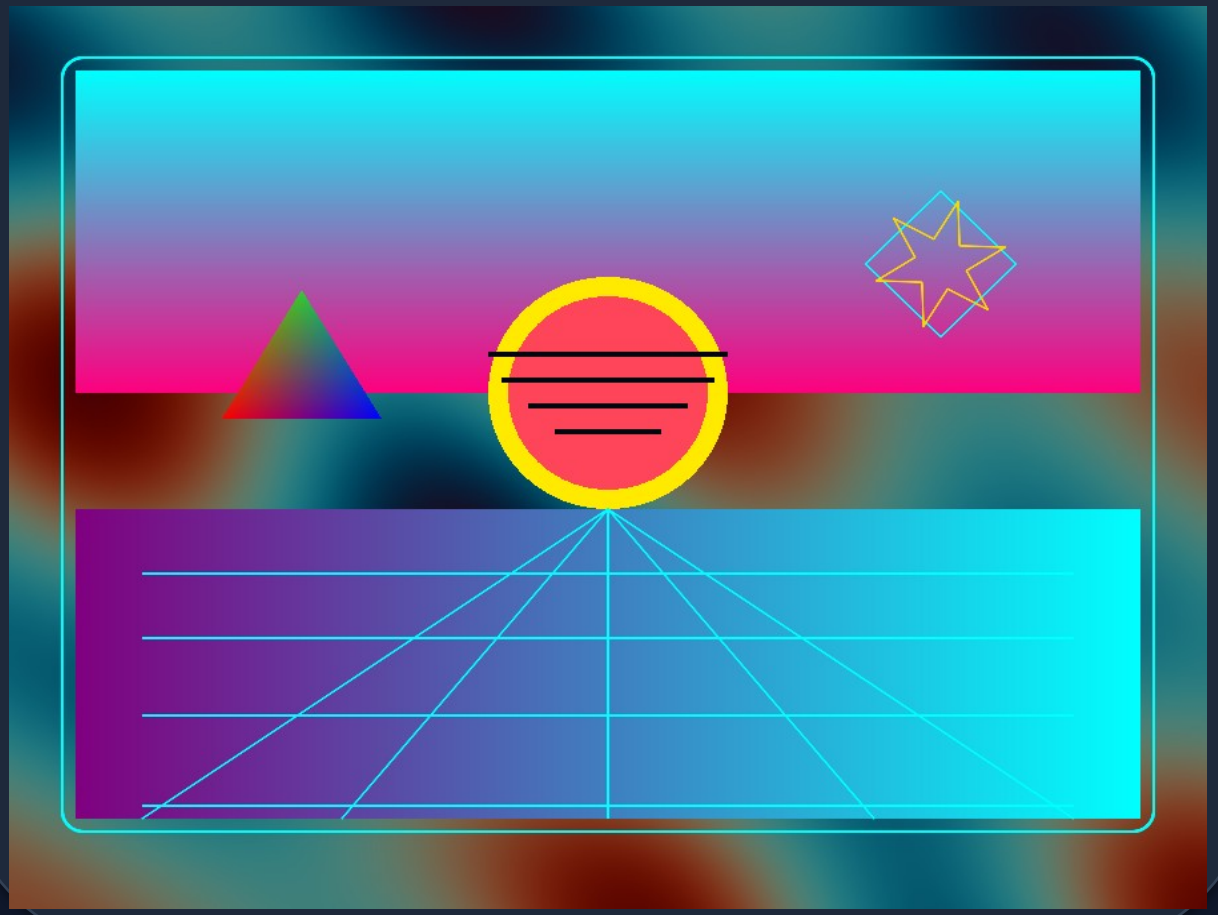
OUTPUT & DEMO

Output Demonstration: 2D Geometry & GPU Shaders

2D Primitives and Shaders

- **2D Vector Geometry**
  Full support for rectangles, rounded rectangles, squares, centered boxes, circles, and ellipses in both filled and outline modes.
- **Color Dictionary**
  35+ built-in named colors (cyan, magenta, gold, lime, coral, crimson) alongside standard hex codes (#RRGGBB).
- **GPU Fragment Shaders**
  Hardware-accelerated GLSL fragment shaders calculating per-pixel animations at 60 FPS (plasma, cyberpunk grid, waves, CRT scanlines).
- **Live Shader Cycling**
  Pressing the Spacebar at runtime dynamically switches active fragment shaders without restarting the application.
- **Launch Command**
  ./mycompiler cyberpunk.my && ./final_game

Running Application: Real-Time GPU Fragment Shader and 2D Primitives



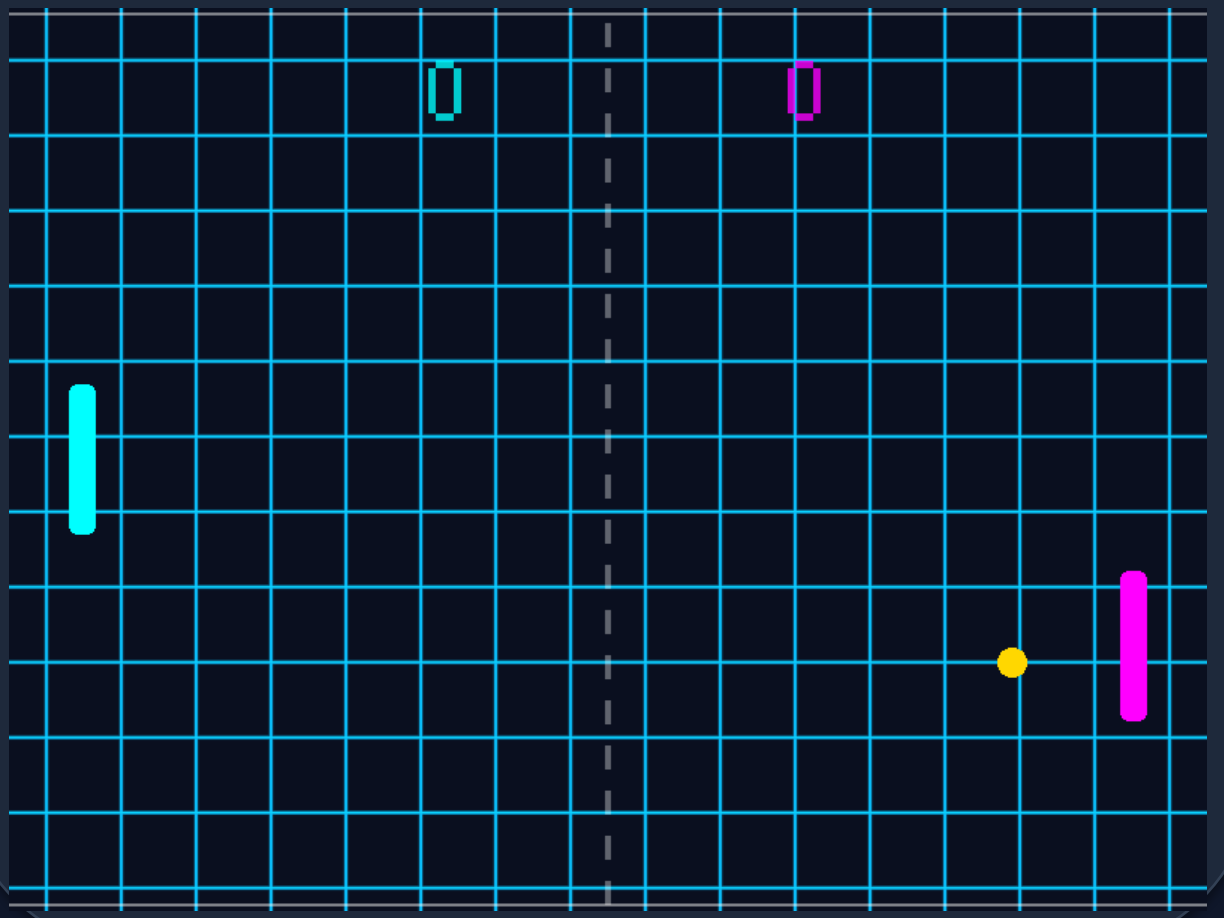
GAME DEMO

Output Demonstration: Interactive 2D Pong Game Engine

Interactive Game Engine (pong.my)

- **Declarative Activation**
  The entire game mode is enabled with a single DSL keyword: pong.
- **Player Controls**
  Player 1 controls the left paddle using W / S or Up / Down keys with boundary clamping.
- **Autonomous AI Opponent**
  The AI opponent tracks the ball position with reaction delay, providing a balanced match.
- **Collision Mathematics**
  Ball-to-paddle detection uses Circle vs. Axis-Aligned Bounding Box (AABB) intersection with angle deflection based on hit position.
- **Real-Time Scoreboard**
  Live retro bitmap font displays player and AI scores with game reset on 'R'.
- **Launch Command**
  ./mycompiler pong.my && ./final_game

Running Application: 2D Pong Game with AI, Scoreboard, and Live Shader



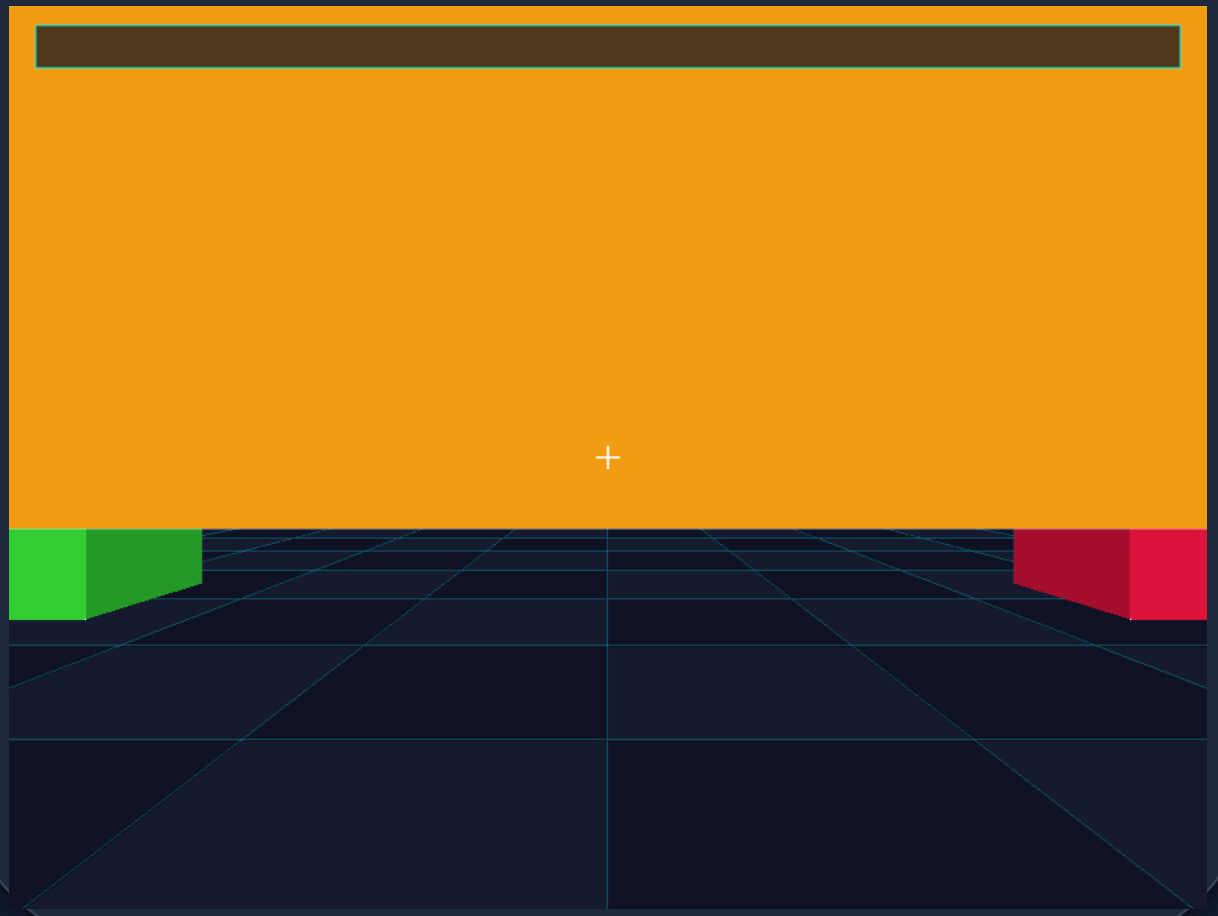
3D ENGINE DEMO

Output Demonstration: 3D First-Person POV Environment

3D World Engine (world3d.my)

- **mode_3d Keyword**
  Switches the pipeline from 2D orthogonal projection to 3D perspective projection with depth-buffering (GL_DEPTH_TEST).
- **First-Person Camera**
  Full 6-degrees-of-freedom navigation: WASD for forward/back/strafe movement, and mouse-look for yaw and pitch.
- **Procedural Environment**
  DSL generates checkerboard floor grids, room boundary walls, monolith pillars, and colored cubes.
- **Axial Wall-Sliding Collision**
  Evaluates X and Z displacement independently to prevent the camera from passing through walls and cubes while preserving smooth sliding.
- **Launch Command**
  ./mycompiler world3d.my && ./final_game

Running Application: 3D First-Person Environment with Depth Buffering



ROADMAP & SUMMARY

Future Implementation and Conclusion

Future Implementation Roadmap

- **3D Lighting Models**
  Implement Blinn-Phong shading with ambient, directional sunlight, and point light calculations in GLSL.
- **3D Model Loader (.OBJ / .GLTF)**
  Enable importing complex 3D meshes into DSL scenes via declarative commands like 'load_mesh "model.obj" x y z'.
- **Sound Engine (SDL_mixer)**
  Integrate retro sound effects for paddle bounces, wall collisions, scores, and ambient background music.
- **Rigid-Body Physics**
  Extend collision detection with mass, gravity, velocity restitution, and multi-body response.
- **Language Constructs**
  Introduce support for user variables, repeat loops (repeat 10 { ... }), and conditional logic.

Project Summary and Deliverables

- **Core Goal Accomplished**
  Successfully proved that declarative DSL scripts can be compiled into native, high-performance OpenGL applications with zero manual boilerplate.
- **Functional Deliverables**
  Includes full 2D graphics suite, 35+ colors, GPU fragment shaders, interactive 2D Pong with AI, and a 3D first-person world engine.
- **Academic Documentation**
  Accompanied by a 14-page formal Word report (Project_Report_Shader_Compiler_DSL.docx) and technical documentation (README.md).
- **Public Repository**
  Code, compiler, engine runtime, and presentations are verified and hosted online at: https://github.com/eldhoGI2/shader-compiler.

Thank you. Ready for live demonstration and questions.